

議事録 — 2025 年 6 月

目次

プロジェクト Orbit 一文書ポータル 1

    プロジェクト概要 . . . . . 1

    背景・目的 . . . . . 2

    システム構成 . . . . . 2

    マイルストーン . . . . . 2

開発ガイド 2

    環境構築 . . . . . 2

    ブランチ運用 . . . . . 3

    コーディング規約 . . . . . 4

    テスト . . . . . 4

API リファレンス 4

    運用計画（パス）管理 . . . . . 4

    衛星リソース管理 . . . . . 6

    エラーコード一覧 . . . . . 6

議事録 — 2025 年 6 月 6

    2025-06-15 開発定例 . . . . . 6

    2025-06-08 設計レビュー . . . . . 7

プロジェクト Orbit 一文書ポータル

プロジェクト Orbit は、小型衛星の運用を担う地上管制システムを新規開発するプロジェクトです。

プロジェクト概要

| 項目      | 内容                          |
|---------|-----------------------------|
| プロジェクト名 | プロジェクト Orbit（小型衛星 地上管制システム） |
| 開始日     | 2025 年 4 月 1 日              |
| リリース予定  | 2026 年 3 月 31 日             |
| PM      | 山田 太郎                       |

| 項目       | 内容    |
|----------|-------|
| リードエンジニア | 鈴木 花子 |

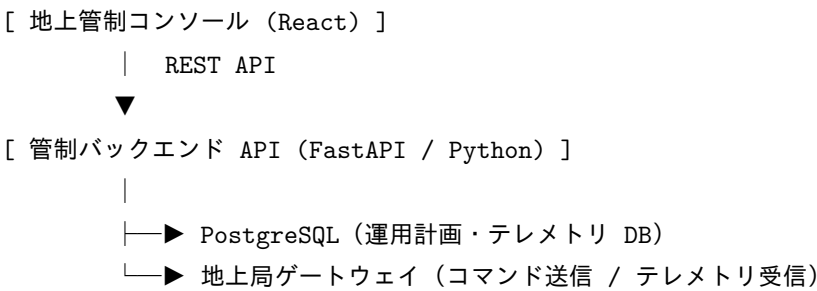
## 背景・目的

現行の衛星運用は個別スクリプトと Excel の運用台帳に依存しており、以下の課題を抱えています。

- Excel ベースの運用台帳によるパス計画のデータ不整合
- テレメトリ確認のリードタイムが長く、異常検知が平均 2 周回ぶん遅れる
- モバイル非対応で当直オペレーターが管制室に拘束される

本プロジェクトではこれらを解消し、リアルタイムなテレメトリ監視・パススケジューリングの自動化・地上局 API 連携を実現します。

## システム構成



## マイルストーン

| フェーズ | 期間                | 成果物                      |
|------|-------------------|--------------------------|
| 要件定義 | 2025/04 ~ 2025/05 | 要件定義書・運用シナリオ             |
| 設計   | 2025/06 ~ 2025/07 | DB 設計書・API 仕様書・地上局 IF 仕様 |
| 開発   | 2025/08 ~ 2025/12 | ソースコード・単体テスト             |
| 検証   | 2026/01 ~ 2026/02 | 結合テスト・運用リハーサル            |
| リリース | 2026/03           | 本番運用開始                   |

**Eleventy** について: この文書は Eleventy (11ty) で管理しています。JavaScript で柔軟にカスタマイズでき、依存ライブラリが最小限です。

## 開発ガイド

### 環境構築

#### 必要なツール

- Python 3.11 以上

- Node.js 20 以上（フロントエンド）
- Docker / Docker Compose
- Git

#### バックエンドのセットアップ

# リポジトリのクローン

```
git clone https://github.com/your-org/project-orbit.git
cd project-orbit/backend
```

# 仮想環境の作成・有効化

```
python -m venv .venv
source .venv/bin/activate
```

# 依存パッケージのインストール

```
pip install -r requirements.txt
```

# 環境変数の設定

```
cp .env.example .env
```

# DB の起動・マイグレーション

```
docker compose up -d db
alembic upgrade head
```

# 開発サーバー起動

```
uvicorn app.main:app --reload
```

#### フロントエンドのセットアップ

```
cd project-orbit/frontend
npm install
cp .env.example .env.local
npm run dev
```

---

## ブランチ運用

```
main          ← 本番リリース済みコード（直接 push 禁止）
├── develop   ← 開発統合ブランチ
│   ├── feature/xxx    機能追加
│   ├── fix/xxx        バグ修正
│   └── docs/xxx       文書更新
```

## Pull Request のルール

1. develop ブランチへの PR を作成
2. レビューアーを 1 名以上アサイン
3. CI (テスト・Lint) が全てグリーンであること

Squash and merge を使用してください。

---

## コーディング規約

### バックエンド (Python)

# 良い例: 型ヒントを必ず付ける

```
async def get_pass(pass_id: str) -> PassResponse:  
    ...
```

- フォーマッタ: ruff format
  - リンタ: ruff check
  - 型チェック: mypy
- 

## テスト

```
pytest -v          # バックエンド  
npm test           # フロントエンド  
npm run test:e2e   # E2E (Playwright)
```

## API リファレンス

ベース URL: `https://api.internal.example/v1`

認証: Bearer トークン (Authorization: Bearer <token>)

---

## 運用計画 (パス) 管理

可視パス一覧取得

GET /passes

クエリパラメータ

| パラメータ        | 型             | 必須 | 説明                                           |
|--------------|---------------|----|----------------------------------------------|
| status       | string        | -  | scheduled / confirmed / executed / cancelled |
| satellite_id | string        | -  | 衛星 ID で絞り込み（例: SAT-001）                      |
| from         | string (date) | -  | AOS（可視開始）日時の開始                               |
| to           | string (date) | -  | AOS 日時の終了                                    |
| limit        | integer       | -  | 取得件数（デフォルト: 20、最大: 100）                      |
| offset       | integer       | -  | オフセット（デフォルト: 0）                              |

## レスポンス例

```
{
  "total": 142,
  "items": [
    {
      "id": "PASS-2025-00142",
      "status": "confirmed",
      "satellite_id": "SAT-001",
      "ground_station": "GS-AKITA",
      "aos_at": "2025-06-15T09:30:00+09:00",
      "max_elevation_deg": 54.8
    }
  ]
}
```

## パス予約作成

POST /passes

## リクエストボディ

```
{
  "satellite_id": "SAT-001",
  "ground_station": "GS-AKITA",
  "aos_at": "2025-06-15T09:30:00+09:00",
  "tasks": [
    { "task_id": "TASK-0032", "priority": 2 }
  ],
}
```

```
"note": "夜間パス・低仰角注意"
}
```

## 衛星リソース管理

テレメトリ照会

GET /telemetry/{satellite\_id}

レスポンス例

```
{
  "satellite_id": "SAT-001",
  "satellite_name": "ひかり 1 号",
  "battery_pct": 87,
  "storage_free_mb": 1240,
  "mode": "nominal",
  "ground_station": "GS-AKITA"
}
```

## エラーコード一覧

| コード              | HTTP | 説明                   |
|------------------|------|----------------------|
| not_found        | 404  | リソースが存在しない           |
| validation_error | 422  | リクエストパラメータが不正        |
| pass_conflict    | 409  | 指定時間帯に地上局が予約済み（パス競合） |
| unauthorized     | 401  | 認証トークンが無効            |
| forbidden        | 403  | 操作権限がない              |

## 議事録 — 2025 年 6 月

2025-06-15 開発定例

日時: 2025 年 6 月 15 日（日）14:00 ~ 15:00

場所: Zoom + 東京オフィス 3F 会議室

参加者: 山田（PM）、鈴木（リードエンジニア）、佐藤（FE）、田中（BE）、木村（QA）

### 議題

#### ■1. スプリント 10 レビュー

- テレメトリ照会 API の実装完了 ☑

- パス予約画面のモバイル対応 完了 ☑
- 可視パス一覧の CSV エクスポート機能 → 残課題あり

## ■2. CSV エクスポートの仕様変更

仕様変更: 運用レビューで以下が決定。

- 最大 10,000 件に制限
- 列順を運用台帳フォーマットに合わせる
- 文字コードは Shift-JIS

担当: 田中、期限: 2025-06-20

## ■3. 認証方式の確認

- IdP: Azure AD (SAML 2.0)
- SP メタデータ URL を IT 部門に連携済み
- テスト環境での検証は 2025-06-18 予定

### アクションアイテム

| # | 内容                                         | 担当    | 期限         |
|---|--------------------------------------------|-------|------------|
| 1 | CSV エクスポート仕様<br>変更の実装                      | 田中    | 2025-06-20 |
| 2 | SSO テスト環境検証                                | 佐藤・田中 | 2025-06-18 |
| 3 | リハーサル DB 週次リ<br>セットスクリプト                   | 木村    | 2025-06-22 |
| 4 | レビュー翌営業日ルール<br>を<br>CONTRIBUTING.md<br>に追記 | 鈴木    | 2025-06-16 |

## 2025-06-08 設計レビュー

日時: 2025 年 6 月 8 日 (日) 15:00 ~ 16:30

参加者: 山田、鈴木、田中

### 決定事項

- パスの重複予約防止は楽観的ロック (`version` カラム) で実装
- パス ID の採番は `PASS-{年}-{5桁連番}` 形式
- 論理削除は `deleted_at` カラムで管理